

LLM-Driven Intent Inference & Hierarchical Goal Modeling in Software Engineering

Foundations of Plan Recognition and Intent Modeling

Understanding an agent's **plans and goals from its actions** has long been studied in AI as *plan recognition* (also called goal or intent recognition) ¹. Classical research formalized plan recognition as inferring an agent's top-level goals and the hierarchical plan structure behind observed behaviors ¹. Early frameworks (e.g. Kautz & Allen 1986) treated plan recognition as a form of logical abduction, identifying which *plan graph* of actions and sub-actions best explains the observations. Modern surveys note that this is a *synergistic research area* combining user modeling, probabilistic reasoning, and machine learning ¹. Importantly, plan recognition algorithms produce **hierarchical interpretations** – they don't just assign a single label to a sequence of actions, but infer an *intention structure* (goals, subgoals, and steps) that could have produced those actions ¹.

A closely related thread is **Hierarchical Task Network (HTN) planning**, which provides a formalism for representing and reasoning about goals and sub-tasks. In HTN planning, complex tasks are broken down into **structured subtasks** repeatedly until primitive executable actions are reached ². Planners rely on *methods* (decomposition recipes) to expand high-level tasks into lower-level steps ³. This yields a *task decomposition tree* (or **intention tree**) capturing how an abstract goal can be achieved via a hierarchy of sub-tasks ² ⁴. The HTN paradigm mirrors human problem-solving and is well-suited to domains where plans naturally form hierarchies ⁵. These ideas are foundational for modeling **developer intents as evolving trees** of goals, since a programming task (like adding a feature) can often be seen as a high-level goal decomposed into sub-tasks (modify module A, update tests, refactor helper functions, etc.).

Notably, *plan-based models* have been applied to software engineering contexts for decades. For example, Huff and Lesser's early work (1988) described a **plan-based intelligent assistant** to monitor software development processes, using plan recognition to interpret a programmer's sequence of actions in terms of higher-level process steps ⁶. This established that AI systems could, in principle, watch a developer's actions (editing code, running tests, etc.) and infer the *process context* (e.g. "implementing a new feature" vs. "debugging a failing test"), enabling the tool to anticipate the developer's needs or catch deviations. Such efforts laid groundwork for viewing software engineering as **intent-driven** behavior that can be recognized and supported by intelligent agents.

Inferring Developer Intent from Code and Natural Language

In software engineering research, there is a rich history of trying to **infer a developer's intent** behind code changes and edit sequences. A common focus has been classifying the *purpose* of code commits. Many studies define **Software Change Intents (SCIs)** as the high-level reasons for a code change – e.g. fixing a bug, adding a feature, refactoring, improving performance, updating documentation, etc. ⁷ ⁸. Early foundational work by Hindle et al. (2009) pioneered automated classification of code changes into maintenance categories (e.g. corrective vs. perfective maintenance) using keyword heuristics and shallow analysis ⁹. This line of work grew into numerous techniques for classifying commits using commit messages and diffs. A recent meta-study of 122 publications on SCIs finds a broad consensus that

knowing the intent behind code changes is valuable for program understanding and software evolution ⁸. However, it also found that prior research is fragmented and lacks a common framework – studies often propose their own taxonomies and methods in isolation, and the practical benefits of systematically tracking change intents are not yet fully realized ⁸. In short, a lot of fundamental groundwork (taxonomy of intents, predictors, etc.) exists, but integrating intent inference deeply into development workflows remains an open challenge.

Traditional techniques for commit intent inference relied on shallow text analysis – for example, looking for keywords in commit messages or code diffs. These **rule-based classifiers** often mislabel the intent; e.g. a commit message saying “improve documentation” might be naively flagged as an “enhancement” because of the word “improve” ¹⁰. Recent advances leverage **Large Language Models (LLMs)** to overcome these limitations. For instance, Oukhay *et al.* introduce an LLM-based Change Classifier (LLMCC) that examines the code differences, commit description, and other context to predict the change’s intent ¹⁰. This general-purpose approach treats intent inference as a natural-language understanding problem, allowing the model to consider semantics rather than just keywords. In evaluations on open-source projects, the LLM-driven classifier significantly outperformed prior heuristic and ML-based methods, improving F1 scores by up to 33% and better identifying critical changes (like bug fixes) for prioritization ¹¹. This demonstrates the promise of **LLM-driven intent inference** in software engineering – by digesting code context and natural language together, LLMs can infer “what the developer is trying to achieve” more accurately than fixed rules.

Another emerging direction is using LLMs to interpret **natural-language instructions or code review comments** and map them to code-edit intents. Guo *et al.* (2025) present an “intention-based code refinement” framework where an AI system first **extracts the reviewer’s intended change** from a review comment, then applies that intention to generate a code modification ¹². They define a set of intention templates (like “rename variable X to Y” or “optimize this loop”) and use a combination of rule-based parsing and LLM prediction to classify a free-form comment into one of these intent categories ¹². Given the extracted intent, a code change is then generated to fulfill it, using an LLM informed by the intent specification. This two-step approach (understand intent → enact intent) yielded notable improvements in automated code refinement: about 79% accuracy on intent extraction and a 66% success rate in applying the code changes, far surpassing end-to-end neural baselines ¹³. Such work is *foundational* in showing how LLMs can be harnessed to bridge the semantic gap between **natural-language goals** and **actual code edits**. It moves beyond treating code changes as independent events, instead viewing them as steps motivated by an explicit intent that the AI can recognize and follow.

Hierarchical Goal Models and Intention Trees

A key aspect of going “beyond classifying intents in sessions” is to represent developer behavior as **evolving, hierarchical intention structures** rather than a flat sequence of unrelated events. Hierarchical planning formalisms provide a theoretical backbone for this. In AI planning, the idea of an *intention structure* or plan decomposition is often represented as a tree (or DAG) of goals and subgoals over time ² ⁴. Software engineering tasks naturally lend themselves to hierarchical modeling: for example, a top-level goal “Implement Feature X” may spawn sub-tasks like “Update Database Schema,” “Modify Backend API,” “Adjust UI,” each of which can further decompose (e.g. “Update Database Schema” might involve “Write migration script” and “Update ORM models”). Fundamentally, this is analogous to an HTN planning problem.

Influential symbolic plan-recognition work has shown how partial observed actions can be parsed into *plan trees*. For instance, hierarchical plan recognition algorithms like GEAPI or PHATT (Geib, Goldman, etc.) use libraries of task schemas to infer possible **plan decompositions** that explain a sequence of

actions. Jun Hong (2000) introduced a graph-based plan recognition approach using a **Goal Graph**: a structure that records observed actions and partial goal states at each time step ¹⁴ ¹⁵. As actions are observed, the Goal Graph is incrementally updated, and analysis of the graph reveals which goals are being achieved or abandoned over time ¹⁶ ¹⁷. Notably, this approach can handle *interleaved and evolving goals*: at any point, some goals may be partially achieved, new goals may appear, and some may be dropped, and the graph-based algorithm can maintain multiple hypotheses in parallel ¹⁶ ¹⁷. This kind of formalism is very relevant to the developer workflow: often a programmer's work is not a linear path to one goal, but rather a juggling of subgoals, adjustments, and even shifts in objective. A **persistent intention memory** would similarly need to track such branching and merging of tasks over a coding session.

There have also been conceptual frameworks arguing for first-class representations of developer intent. *Intentional Programming*, pioneered by Simonyi in the 1990s, posited that source code could be represented in terms of **intention nodes** in a persistent tree structure ¹⁸. In Simonyi's vision, a developer would operate directly on the intentions (e.g. "define a high-level concept" or "apply a design pattern") and the system would maintain an *executable representation* of those intentions, allowing the underlying code to be regenerated or transformed as needed. This led to an experimental environment where the entire codebase was stored as ~1.7 million *intention instances* forming a graph of abstractions ¹⁹. The goal was to make software "immortal" by preserving the *why* (the intention) separately from the textual implementation ¹⁸. While Intentional Programming in practice did not see widespread adoption, it stands as an influential paradigm: it introduced the idea of an **intention tree that is continually revised and elaborated** as development progresses, with the code being a projection of that intention structure. Modern takes on this idea could enable an AI assistant to maintain an *HTN-like model* of a developer's tasks, updating it as the developer writes code or as requirements change.

Maintaining a hierarchical intention model unlocks capabilities like **drift detection** and **tangled task disentanglement**. If the system knows the current intention hierarchy, it can detect when the developer's recent actions start diverging from the expected plan (possibly indicating requirement drift or a misunderstood goal). It can also identify when two unrelated intents are being interleaved. Tangled work is a well-known problem in version control: developers often bundle *unrelated changes* (for example, a bug fix *and* an opportunistic refactoring) into one commit, creating what researchers call a *tangled commit* ²⁰. Identifying and untangling such intermixed intents has seen foundational research. Dias *et al.* (2015) collected data on developers who carefully split their work into atomic changes, and introduced *EpiceaUntangler*, a tool that uses fine-grained code change graphs to automatically cluster code edits by intended task ²⁰ ²¹. Their approach could successfully partition a stream of changes into separate intent-based clusters about 75% of the time on average ²¹. This kind of technique, especially if guided by a higher-level understanding of the developer's goals, could be integrated into AI-assisted workflows: the system might warn "It looks like you are mixing two tasks: refactoring X and adding feature Y – consider separating these," or even help perform the separation. Achieving this requires the system to **attribute each code edit to a specific intent** in the intention tree, rather than treating all edits as one amorphous activity.

Mining the "Why": Software Change Rationale

An orthogonal but crucial facet is building an *auditable "why" layer* for software changes – capturing the rationale behind changes in a structured, queryable form. In practice, the primary vehicle for recording rationale has been commit messages and issue trackers. In high-quality projects (e.g. the Linux kernel), developers are encouraged to **explain the motivation and impact** of each commit ²². These messages often contain nuggets of "why" (e.g. "fix null pointer exception when user is not logged in" explains the reason for a code fix). Researchers have mined such data to analyze how well rationale is documented. For example, Tian *et al.* and Li *et al.* (cited in Dhaouadi *et al.* 2023) examined thousands of

commit messages to measure the presence of *what* versus *why* information ²³. They found that many commits lack explicit rationale, highlighting a gap between the latent intent and what gets documented.

To address this, recent work has tried to explicitly **decompose and categorize change rationale**. An influential study by Al-Safwan and Servant (2019) asked *developers themselves* how they describe the rationale of code commits ²⁴ ²⁵. The result was that developers consider a **multi-faceted rationale** – not a single “reason”, but up to 15 *distinct components* of rationale for each change ²⁵. These components include the *goal* of the change, the *need* or problem being addressed, the *benefits* expected, *constraints* or requirements, *alternatives* considered (and which one was chosen), *dependencies* affected, who the *committer* was, *time* and *location* of the change, *modifications* made, an explanation of *why those modifications* were done that way, how the change was *validated* (e.g. tests), the *maturity stage* of the change, and any *side effects* ²⁵. This decomposition provides a conceptual **framework for a “why” layer**: instead of treating rationale as a monolithic text, it can be structured into fields. A tool could then prompt a developer (or use AI) to fill in these fields when making a commit. Having an *explicit rationale model* would make the intention memory auditable and searchable – future developers (or AI assistants) could query not just “what does this code do?” but “why was this code written this way, what alternatives were rejected, and what constraints were at play?”.

Mining rationale also extends to using developer activity traces beyond text. For instance, one could infer rationale from **test cases and bug reports**: if a failing test is observed and then code is changed until the test passes, the system can conclude the intent was “fix the bug revealed by that test.” Tying code edits to triggering events (like test failures, profiling results, user stories) can enrich the intention model with *links to the causes*. Some research in mining software repositories has looked at linking commits to issues or requirements, essentially connecting the code change (the “how”) to the higher-level ticket (the “what” and “why”). An auditable intent framework would incorporate these links, providing traceability. For example, Murphy *et al.* (2014) proposed retroactive links between code commits and task descriptions to recover context – essentially attaching the “issue” to the code changes that resolved it. While a fully automated “why layer” is still a vision, the foundational research (on commit intent classification, rationale analysis, traceability, etc.) is laying the bricks for it.

Toward Intentional Development Environments

Bringing these threads together, we see an exciting vision forming: **AI-assisted development environments that maintain a live, hierarchical model of the developer’s intentions**, continuously updated via intent inference from code and conversation, and used to guide the developer’s workflow. Such an environment would leverage LLM-driven intent recognition (as seen with LLMCC and others) to classify the likely intent of each code edit or natural-language instruction. It would use hierarchical plan recognition techniques to organize these intents into an “intention tree” – essentially the AI’s current belief of the developer’s game plan. This intention tree would be **persistent and revisable**: as the developer clarifies requirements or changes direction, the tree is adjusted (much like a GPS recalculating a route when you deviate). If the developer’s actions start to drift from the current plan (e.g. they begin editing an unrelated module with no obvious connection to the active task), the system can detect this drift and either recognize a new subgoal or warn of potential distraction.

Throughout, the system also logs the **rationale** for each intention in the tree, drawing from commit messages, issue discussions, and even the developer’s interaction history. The result is a rich context for each code change: not just “what changed,” but “what goal it fulfilled, how it fit into larger goals, and why it was done that way.” This could fundamentally improve activities like code review, debugging, and maintenance. For instance, an *intent-aware refactoring recommendation* could be made: “You introduced

Feature X under a tight deadline, but your intention log notes a *deferred subtask* to refactor the error handling – shall I remind you to do that now?” The AI, knowing the “**why**” and “**what’s next**”, can provide much more pertinent suggestions than a syntax-based linter.

While much of this integrated vision remains aspirational, the **building blocks are in place**. Foundational AI planning work gives us formalisms for intention hierarchies and algorithms for goal recognition. Software engineering research provides taxonomy of intents and empirical understanding of how developers express intent and rationale. And modern LLM-based techniques supply the raw capability to interpret complex NL+code contexts. The coming together of these threads is evident in conceptual works like Microsoft’s recent *Intent Prediction Networks* (which aim to anticipate developer needs by recognizing intent patterns) ²⁶. In summary, the field is moving beyond treating code changes as isolated events or classifying them with flat labels, and toward maintaining a **holistic, evolving model of developer intentions**. This model, akin to an “*intention tree memory*,” will enable smarter automation in coding assistants – from untangling mixed tasks ²⁰, to detecting requirement drift, to providing explanations for code (why was this done?) on demand. The research highlighted here – spanning plan recognition in AI, intent inference in SE, hierarchical planning, and rationale mining – forms the backbone of this emerging paradigm of **intent-aware software development**.

Sources:

- Gita Sukthankar *et al.* (2014). *Plan, Activity, and Intent Recognition* – Introduction ¹.
- GeeksforGeeks (2026). *Hierarchical Task Network Planning in AI* ² ⁴.
- Microsoft Research (2021). *Large-Scale Intent Analysis for Code Changes* – Wang *et al.* ⁷.
- Yi Li *et al.* (2024). *A Meta-Study of Software-Change Intentions* – ACM Computing Surveys ⁸.
- ÉTS Montréal (2025). *LLM Change Intent Classifier (LLMCC) article* – Oukhay & Ouni ¹⁰ ¹¹.
- Qi Guo *et al.* (2025). “*Intention is All You Need*” – *Intention-based Code Refinement* ¹² ¹³.
- Martín Dias *et al.* (2015). *Untangling Fine-Grained Code Changes* – atomic commits and EpiceaUntangler ²⁰ ²¹.
- Mouna Dhaouadi *et al.* (2023). *CoMRAT: Commit Message Rationale Analysis Tool* ²².
- Khadijah Al Safwan & Francisco Servant (2019). *Decomposing the Rationale of Code Commits* ²⁵.
- Charles Simonyi (1995). *Intentional Programming* – Technical Report MSR-TR-95-52 ¹⁸ ¹⁹.

¹ ial.eecs.ucf.edu

<https://ial.eecs.ucf.edu/pdf/PAIRBook-Intro.pdf>

² ³ ⁴ ⁵ Hierarchical Task Network (HTN) Planning in AI - GeeksforGeeks

<https://www.geeksforgeeks.org/artificial-intelligence/hierarchical-task-network-htn-planning-in-ai/>

⁶ [PDF] AI Techniques in Software Engineering - MICE - Columbia University

<https://mice.cs.columbia.edu/getTechreport.php?techreportID=1260&format=pdf&>

⁷ Large-Scale Intent Analysis for Identifying Large-Review-Effort Code Changes - Microsoft Research

<https://www.microsoft.com/en-us/research/publication/large-scale-intent-analysis-for-identifying-large-review-effort-code-changes/>

⁸ Paper accepted by ACM Computing Surveys – Yi Li | Associate Professor | NTU

<https://liyiweb.com/posts/paper-accepted-by-csur-intention/>

⁹ ¹⁰ ¹¹ Large Language Models for Code Change Intent Classification - ÉTS Montréal

<https://www.etsmtl.ca/en/news/grands-modeles-langage-classification-intentions-changement-code>

12 13 **Intention is All You Need: Refining Your Code from Your Intention**

<https://arxiv.org/html/2502.08172v1>

14 15 16 17 **Graph Construction and Analysis as a Paradigm for Plan Recognition**

<https://cdn.aaai.org/AAAI/2000/AAAI00-119.pdf>

18 19 **The Death of Computer Languages, The Birth of Intentional Programming - Microsoft Research**

<https://www.microsoft.com/en-us/research/publication/the-death-of-computer-languages-the-birth-of-intentional-programming/>

20 21 **[1502.06757] Untangling Fine-Grained Code Changes**

<https://arxiv.org/abs/1502.06757>

22 23 **CoMRAT: Commit Message Rationale Analysis Tool** This research work is funded by the Fonds de Recherche du Québec (B2X).

<https://arxiv.org/html/2506.10986v1>

24 25 **Decomposing the Rationale of Code Commits: The Software Developer's Perspective**

https://fservant.github.io/papers/AlSafwan_Servant_ESECFSE19.pdf

26 **The Shift from Software Engineering to "Intent Engineering": Reimagining Human-Computer Collaboration in the Age of AI**

<https://www.linkedin.com/pulse/shift-from-software-engineering-intent-reimagining-ai-andre-3ldde>